

CS 308P: Large Application Practicum

Course Code	CS 308P
Course Name	Large Application Practicum
L-T-P-C	0-0-3-2
Offered in	B.Tech CSE
Approval	31st BoA (Sep 2019), in the B.Tech CSE curriculum; 23rd Senate (Oct 2019), in the B.Tech CSE curriculum

This course follows the syllabus of CS 308, reproduced below.

Course Code	CS 308
Course Name	Large Applications Practicum
L-T-P-C	0-0-3-2
Prerequisites	IC 150
Intended for	UG
Distribution	Compulsory for CSE; CS elective for EE and ME
Semester	6th
Approval	9th Senate (May 2016)

Course Contents

A few lab lectures (8 hours spread over the semester):

- Overview of the Make utility (Makefiles, writing rules, use of variables, Conditionals, Functions, running make)
- Source code revision control (version control basics, introduction to some basic version control systems like CVS, SVN, and Git)
- Introduction to document generation from annotated source code (with specific focus on Doxygen, TwinText, and Natural Docs)
- Software testing (introduction, need for software testing, types of tests, test case design)
- Software testing tools (introduction to some basic testing tools for object-oriented languages e.g., Jester for JUnit)
- Introduction to conventions of fast parsers for context-free grammars: Flex and Bison
- Overview of diagrams in the UML notation (also, how UML translates to programming structures in certain object-oriented languages like C++/Java)
- Introduction to software reverse engineering

Lab assignments (listed below) require 3 hours in the lab, preceded by at least 3 hours at home. The weekly assignments would be targeted at mastering the concepts covered weekly in the course:

- Week 1-2 Use of Make and Makefiles for object-oriented programming languages.
- Week 3-4 Use of a versioning system (e.g., Git, SVN).
- Week 5-6 Document generation from annotated source code using one of the open-source software (e.g., Doxygen, Natural Docs).
- Week 7-8 Software testing and test-case design; use of open-source software testing tools to test object-oriented code (e.g., Jester).
- Week 9-10 Use of parsers for parsing context-free grammars (using Flex/Bison)
- Week 11-12 Application of UML notation and diagrams for object-oriented design.
- Week 13 Reverse engineering of an existing open-source application (using certain reverse engineering tools)

References

1. GNU 'make' pages (<http://www.gnu.org/software/make/manual/make.html>)
2. Boston University's Make Tutorial (<http://www.cs.bu.edu/teaching/cpp/writing-makefiles/>)
3. Emory University's Make Tutorial (<http://www.mathcs.emory.edu/~cheung/Courses/255/Syllabus/1-C-intro/make.html>)
4. Introduction to CVS, SVN, and Git (http://www.linuxdevcenter.com/pub/a/linux/2002/01/03/cvs_intro.html/; <http://git-scm.com/>)
5. Manual pages for Doxygen and Natural Docs (<http://www.stack.nl/~dimitri/doxygen/>; <http://www.naturaldocs.org/>)
6. Paul Ammann and Jeff Offutt, Course code: CS 546, Cambridge University Press, 2008.
7. Carnegie Mellon University resources on Software Testing Tools (https://www.ece.cmu.edu/~koopman/des_s99/sw_testing/; <http://mcahelpline.com/tutorials/testing/testing.pdf>)
8. Andreas Spillner, Tilo Linz, Hans Schaefer, **Software Testing Foundations: A Study Guide for the Certified Tester Exam** (Rock-nook Computing), Rocky Nook, 2011.
9. Flex & Bison, **Text Processing Tools**, John Levine, O'Reilly Media, 2009.
10. GNU resources on Flex and Bison (<http://www.gnu.org/software/bison/>; <http://flex.sourceforge.net/>)
11. UML and object-oriented design (<http://www.agiledata.org/essays/objectOrientation101.html>)
12. Jesse Liberty, Vishwajit Aklecha, **C++ Unleashed**, 1998.
13. Eldad Eilam, **Reversing: Secrets of Reverse Engineering**, Wiley, 2005.